# Processor Verification
# Final Project: Greatest Common Divisor

Lorenzo Pattaro Zonta

June 21, 2026

# 1 Design Under Test - Description

The GCD module computes the Greatest Common Divisor of two unsigned integers utilizing the subtractive Euclidean algorithm. It processes one pair of inputs at a time and interfaces with other components via two independent ready and valid handshake channels.

## 1.1 Mathematical Definition

For unsigned operands $A$ and $B$, the module implements the following mathematical properties:

- $\gcd(A, 0) = A$
- $\gcd(0, B) = B$
- $\gcd(0, 0) = 0$
- $\gcd(A, A) = A$
- $\gcd(A, B) = \gcd(A - B, B)$ when $A > B$
- $\gcd(A, B) = \gcd(A, B - A)$ when $B > A$

## 1.2 Interface Definition

The module is parametrized with `WIDTH`, which defines the width of the data operands and defaults it to 16.

### 1.2.1 Global Signals

- `clk`: Clock signal.
- `rst_n`: Active-low synchronous reset.

### 1.2.2 Input Channel

A transaction is captured on the rising edge where `in_valid` and `in_ready` are simultaneously asserted.

- `in_valid`: Asserted by the producer when `a_in` and `b_in` hold valid data.
- `in_ready`: Asserted by the module when ready to accept a new transaction; high only in the `IDLE` state.
- `a_in`, `b_in`: `WIDTH`-bit operands.

### 1.2.3 Output Channel

A transaction completes on the rising edge where `out_valid` and `out_ready` are simultaneously high.

- `out_valid`: Asserted by the module when `gcd_out` holds a valid result; high during the `DONE` state.
- `out_ready`: Asserted by the consumer when it is ready to accept the result.
- `gcd_out`: `WIDTH`-bit GCD result, which remains stable and valid for the entire period `out_valid` is asserted.

## 1.3 Functional Description and State Machine

The module behaves according to a Finite State Machine (FSM) comprising three states:

- **IDLE**: The module waits for input and asserts `in_ready = 1`.
- **RUN**: The module performs iterative subtraction. In each cycle, if `a_reg > b_reg`, it computes `a_next = a_reg - b_reg`. If `b_reg > a_reg`, it computes `b_next = b_reg - a_reg`.
- **DONE**: The result is ready, and the module asserts `out_valid = 1`.

#### 1.3.1 Transitions

- **IDLE** $\rightarrow$ **DONE**: Occurs in exactly 1 cycle upon an input handshake if the result is known immediately (`a_in == 0`, `b_in == 0`, or `a_in == b_in`).
- **IDLE** $\rightarrow$ **RUN**: Occurs upon an input handshake if operands are non-zero and unequal.
- **RUN** $\rightarrow$ **DONE**: Fires when the next-cycle values of the working registers become equal (`a_next == b_next`).
- **DONE** $\rightarrow$ **IDLE**: Triggered by the output handshake (`out_valid == 1` and `out_ready == 1`).

### 1.4 Latency and Throughput

The module processes one transaction at a time. Latency is defined as the number of cycles from the input handshake edge to the first cycle where `out_valid` is asserted.

- **Early Exit**: 1 cycle (for operands yielding an immediate result).
- **General Case**: $1 + N$ cycles, where $N \geq 1$ represents the number of subtraction steps required for convergence.

### 1.5 Signals table

| Name | Direction | Width | Description |
|---|---|---|---|
| `clk` | Input | 1 | Clock signal |
| `rst_n` | Input | 1 | Active-low sync reset |
| `in_valid` | Input | 1 | Asserted by the producer when a_in and b_in hold valid data |
| `in_ready` | Output | 1 | Asserted by the module when it can accept a new transaction. High only in the IDLE state. |
| `a_in` | Input | WIDTH | First operand |
| `b_in` | Input | WIDTH | Second operand |
| `out_valid` | Output | 1 | Asserted by the module when gcd_out holds a valid result. High for the entire DONE state. |
| `out_ready` | Input | 1 | Asserted by the consumer when it is ready to accept the result. |
| `gcd_out` | Output | WIDTH | GCD result. Valid and stable for the entire period that out_valid is asserted. |

Table 1: GCD Module Input and Output Signals

## 2 Traceability Matrix

| Req ID | Requirement | Feature | Test ID |
|---|---|---|---|
| REQ-001 | GCD is performed correctly for operands A and B | GCD Functioning (General case) | `gcd_random_test` |

Continued on next page

Table 2: Traceability Matrix: Features and Tests (Continued)

| Req ID | Requirement | Feature | Test ID |
|---|---|---|---|
| REQ-002 | Edge cases GCD(0,0), GCD(A, 0) (with $A \neq 0$) and GCD(0, B) (with $B \neq 0$) are covered. Output is correct | GCD Functioning (Edge cases) | `gcd_directed_test` |
| REQ-003 | Edge case GCD(A, A) is covered. Output is correct | GCD Functioning (Edge cases) | `gcd_directed_test` |
| REQ-004 | Edge case GCD(A, B), where $A == 2^{WIDTH} - 1$ or $B == 2^{WIDTH} - 1$ | GCD Functioning (Edge cases) | `gcd_directed_test` |
| REQ-005 | GCD(A,B) is performed correctly for $A > B$ and $B > A$ | GCD Functioning (General case) | `gcd_random_test` |
| REQ-006 | When `rst_n` is low, shift to the IDLE state | Reset behaviour | SVA: `assert_reset_behavior` |
| REQ-007 | When `rst_n` is low, clear internal registers to 0 | Reset behaviour | SVA: `assert_reset_behavior` |
| REQ-008 | When `rst_n` is low, sets `in_ready = 1`, `out_valid = 0`, and `gcd_out = 0`. | Reset behaviour | SVA: `assert_reset_behavior` |
| REQ-009 | Transactions captured only when `in_ready` && `in_valid` are high | Handshake Protocol | SVA: `assert_in_valid_no_drop` and `assert_out_valid_no_drop` |
| REQ-010 | State transitions from IDLE to DONE takes only 1 cycle when $A == 0$ or $B == 0$ or $A == B$ | 1 cycle GCD (Latency) | `gcd_directed_test` |
| REQ-011 | General case of $N \geq 1$ subtraction steps occur in $1 + N$ cycles | $1 + N$ Cycles (Latency) | `gcd_random_test` |
| REQ-012 | `gcd_out` remains stable until `out_ready` is asserted | Output Stability | SVA: `assert_output_stable` |
| REQ-013 | DUT accepts input on handshake and immediately drops `in_ready` to indicate processing has started | FSM Transitions | SVA: `assert_in_ready_drop` |
| REQ-014 | DUT asserts `out_valid` within the maximum theoretical latency limit ($2^{WIDTH}$ cycles) without hanging | FSM Transitions | Procedural watchdog: `MAX_TIMEOUT` |
| REQ-015 | DUT drops `out_valid` on the clock cycle immediately following the `out_ready` handshake. | FSM Transitions | SVA: `assert_out_valid_drop` |
| REQ-016 | Module is ready to accept input (`in_ready == 1`) immediately on the first rising edge after `rst_n` is released, and it is in IDLE state | Post-Reset | SVA: `assert_reset_behavior` |
| REQ-017 | New input handshake can occur on the earliest cycle immediately after an output handshake | Throughput | `gcd_random_test` |

Continued on next page

Table 2: Traceability Matrix: Features and Tests (Continued)

| Req ID | Requirement | Feature | Test ID |
|---|---|---|---|
| REQ-018 | On `in_valid == 1`, `a_in`, and `b_in` must remain stable while stalled (`in_ready == 0`) | Protocol | SVA: `assert_input_stable` |

Table 2: Traceability Matrix: Features and Tests

| Req ID | Test ID | Coverage Goal |
|---|---|---|
| REQ-001 | `gcd_random_test` | 100% Functional Coverage |
| REQ-002 | `gcd_directed_test` | 100% Functional Coverage |
| REQ-003 | `gcd_directed_test` | 100% Functional Coverage |
| REQ-004 | `gcd_directed_test` | 100% Functional Coverage |
| REQ-005 | `gcd_random_test` | 100% Functional Coverage |
| REQ-006 | SVA: `assert_reset_behavior` | 100% Code Coverage and Assertion Pass |
| REQ-007 | SVA: `assert_reset_behavior` | 100% Code Coverage and Assertion Pass |
| REQ-008 | SVA: `assert_reset_behavior` | 100% Code Coverage and Assertion Pass |
| REQ-009 | SVA: `assert_in_valid_no_drop` and `assert_out_valid_no_drop` | 100% Assertion Pass |
| REQ-010 | `gcd_directed_test` | 100% Scoreboard Match |
| REQ-011 | `gcd_random_test` | 100% Scoreboard Match |
| REQ-012 | SVA: `assert_output_stable` | 100% Assertion Pass |
| REQ-013 | SVA: `assert_in_ready_drop` | 100% Assertion Pass |
| REQ-014 | Procedural watchdog: `MAX_TIMEOUT` | Zero UVM Errors (No timeout) |
| REQ-015 | SVA: `assert_out_valid_drop` | 100% Assertion Pass |
| REQ-016 | SVA: `assert_reset_behavior` | 100% Assertion Pass |
| REQ-017 | `gcd_random_test` | 100% Scoreboard Match & Zero UVM Errors |
| REQ-018 | SVA: `assert_input_stable` | 100% Assertion Pass |

Table 3: Traceability Matrix: Coverage Goals

Note: SVA stands for System Verilog Assertions.

## 2.1 Verification Strategy

The Verification Strategy will be divided on different approaches, each one will be described in the following sections.

### 2.1.1 Constrained Random Testing

To tackle the GCD Functioning, a Constrained Random Testing Verification will be implemented. The testbench will create randomized input operands for A and B and a UVM scoreboard, containing a Golden Reference Model that will compute the expected GCD result, will be used to make a comparison with the DUT output. In fact, the scoreboard will collect the randomized inputs, compute the expected GCD result of the two operands and compare it with the DUT output.

### 2.1.2 Directed Testing

To cover GCD functioning corner cases, the early exit scenario (1 cycle GCD Latency) and reset behaviours, I will implement a Directed Testing Verification setup, to correctly cover these particular scenarios.

For the GCD part, I will create specific values for the operands A and B to cover the following cases:

- GCD(0, 0)
- GCD(A, 0) (with $A \neq 0$)
- GCD(0, B) (with $B \neq 0$)
- GCD(A, A)
- GCD(A, B) where $A == 2^{WIDTH} - 1$ or $B == 2^{WIDTH} - 1$

Using the same scoreboard setup, I will create specific tests to cover the early exit scenario, where the GCD is expected to be computed in 1 cycle.

Lastly, I will create a specific test to cover the reset behaviour, where the DUT will be reset and the output will be checked to verify that it is in the expected state.

To check all of these tests, I will embed in the UVM scoreboard previously mentioned to compare the DUT output with the expected output.

### 2.1.3 Assertion Based Verification

Assertions are ideal for checking handshake signals and protocols compliance. In fact, they evaluate over multiple simulation cycles. That is why I decided to implement an Assertion Based Verification strategy using System Verilog Assertions to cover temporal properties, control logic and compliance of the protocols (such as correctness of handshakes). The following properties are tested with this approach:

- handshake protocol for FSM transitions and data transfer (both input and output channels)
- End to end handshake protocol compliance and timeout limits (verifying implicitly from the outside that the internal FSM does not hang or enter deadlocks)
- data stability during stalled input handshake (`a_in` and `b_in` remain stable when `in_valid == 1` and `in_ready == 0`)
- output stability and correct behaviour during reset

These properties will be evaluated during the run of the UVM simulation. However, they are designed to be formal ready, meaning that can be used in a formal verification tool to prove the state space of the control logic without any change.

#### 2.1.4 Summary table

To summarize all of these verification strategies, the following table shows the mapping between the requirements and the verification strategy used to verify them.

| Req IDs | Verification Strategy | Target Features |
|---|---|---|
| REQ-001, REQ-005, REQ-011, REQ-017 | Constrained Random Testing | GCD Functioning (General Case), 1+N Cycles (Latency), Throughput |
| REQ-002, REQ-003, REQ-004, REQ-006, REQ-007, REQ-008, REQ-010 | Directed Testing | GCD Functioning (Edge Cases), 1 Cycle GCD (Latency), Reset Behaviour |
| REQ-009, REQ-012, REQ-013, REQ-014, REQ-015, REQ-016, REQ-018 | Assertion Based Verification | Handshake Protocol, Protocol, FSM Transitions, Output Stability, Post-Reset, |

Table 4: Verification Strategy Summary

## 2.2 Closure Criteria

To consider the verification process complete and successful, the following quantitative metrics must be achieved:

- **100% Test Pass Rate:** All UVM tests (bring-up, directed, and constrained-random) must pass successfully without any fatal errors or timeouts.
- **100% Functional Coverage:** All defined covergroups and coverpoints (including mathematical edge cases, mathematical states, and operand categories) must reach 100% coverage.
- **High Code Coverage:** Achieve at least 95% Line, Branch, and Toggle coverage. Any uncovered code must be manually reviewed and justified (e.g., unreachable code).
- **100% Assertion Pass Rate:** Zero assertion failures during all test runs, confirming protocol and timing compliance.

## 2.3 UVM Architecture

For the GCD module, I implemented a UVM testbench architecture. One methodology could be of implement two different agents, one for input channel and one for the output one, as they have independent handshake protocols.

However, I decided to implement a single agent, because both channels share the same clock and reset signals, and the dimension of the module itself is relatively small.

The following diagram shows the UVM architecture that I implemented.

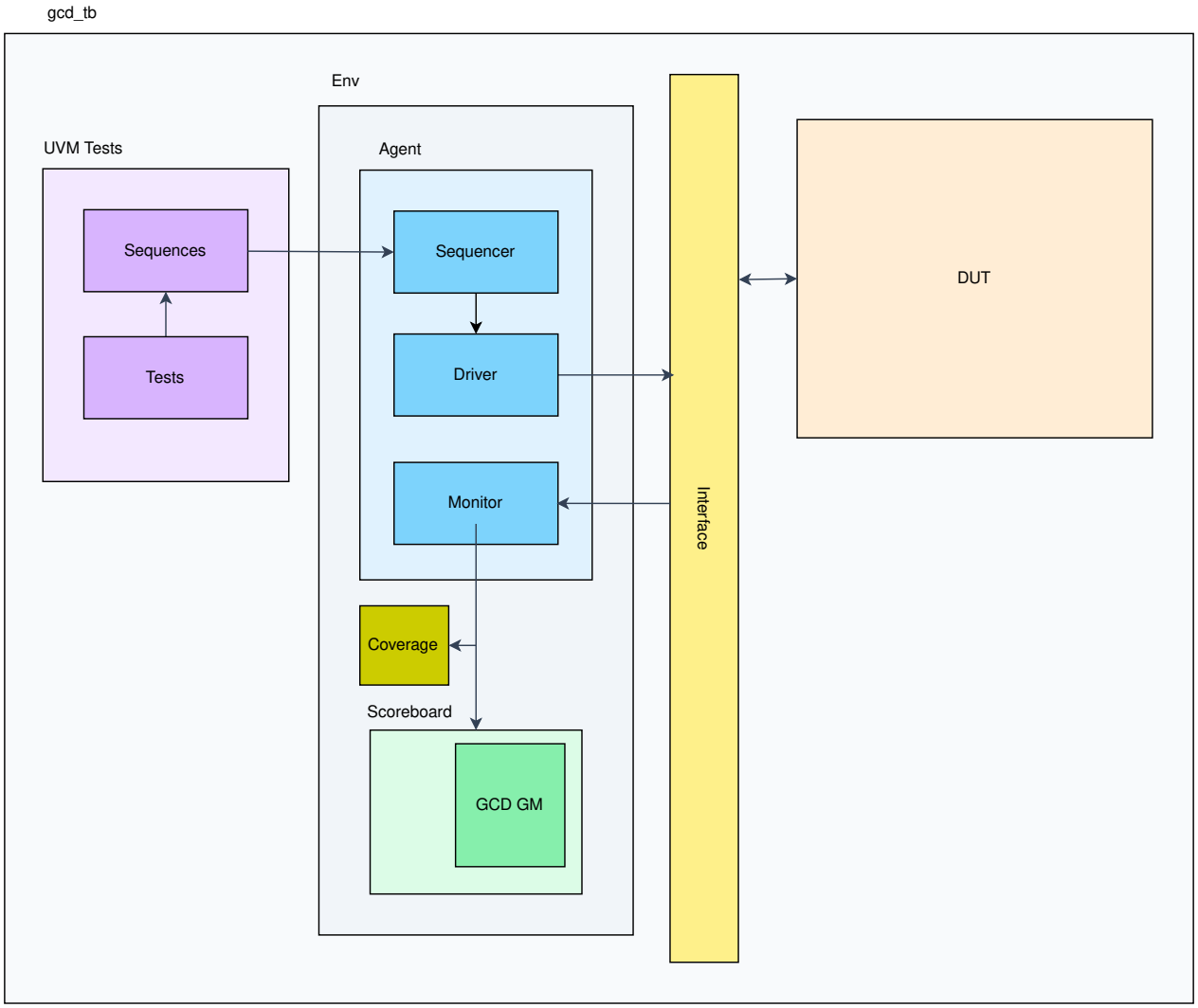


Figure 1: UVM testbench architecture diagram

# 3 Conclusion

# 4 Appendix

To show the waveforms, I have used the Surfer software.

## 4.1 Number of hours required

For the completion of this laboratory, it has been needed x hours.

## 4.2 Use of AI

Gemini AI has been used to fix bugs in the code and to improve some sentences in the report.

## 4.3 Appendix A